~\Downloads\Inventory_env_class.py

```python
"""Mini Project 2 Part 1 - Gymnasium inventory environment (aligned version).

Actions: 0 wait, 1 purchase ONE raw unit, 2 convert TWO raw units into ONE product.
The six state features are unchanged. Demand in the observation is the demand
that will be served after the current action. Demand is 0 or 1 unit per period;
its probability varies stochastically around demand_mean.

Reward is an operating performance score, NOT ending cash or accounting profit.
Only purchases and sales change cash; holding/stockout penalties affect reward.
"""
import random
from collections import deque

import gymnasium as gym
from gymnasium import spaces
import numpy as np
import torch
from torch import nn


class InventoryManagementEnv(gym.Env):
    metadata = {"render_modes": ["human"]}

    def __init__(self, initial_cash=1000.0, demand_mean=0.25,
                 demand_volatility=0.10, raw_price_base=5.0,
                 product_price_base=20.0, price_std=0.2,
                 holding_cost=0.1, stockout_penalty=5.0,
                 invalid_action_penalty=2.0, bankruptcy_penalty=100.0,
                 conversion_bonus=1.0, max_steps=500):
        super().__init__()
        if not 0 <= demand_mean <= 1 or demand_volatility < 0:
            raise ValueError("Demand mean must be a probability in [0,1], volatility >= 0")
        if max_steps <= 0:
            raise ValueError("max_steps must be positive")
        self.action_space = spaces.Discrete(3)
        self.observation_space = spaces.Box(
            low=np.zeros(6, dtype=np.float32),
            high=np.full(6, np.inf, dtype=np.float32), dtype=np.float32)
        self.initial_cash = initial_cash
        self.demand_mean = demand_mean
        self.demand_volatility = demand_volatility
        self.raw_price_base = raw_price_base
        self.product_price_base = product_price_base
        self.price_std = price_std
        self.holding_cost = holding_cost
        self.stockout_penalty = stockout_penalty
        self.invalid_action_penalty = invalid_action_penalty
        self.bankruptcy_penalty = bankruptcy_penalty
```

```
        self.conversion_bonus = conversion_bonus
        self.max_steps = max_steps
        self.current_step = 0
        self.last_product_inventory_before_sale = 0
        self.last_sold_units = 0

    def _sample_demand(self):
        probability = np.clip(
            self.np_random.normal(self.demand_mean, self.demand_volatility), 0, 1)
        return int(self.np_random.binomial(1, probability))

    def reset(self, *, seed=None, options=None):
        super().reset(seed=seed)
        self.raw_inventory = 0
        self.product_inventory = 0
        self.cash = float(self.initial_cash)
        self.raw_price = max(0.1, self.np_random.normal(self.raw_price_base, self.price_std))
        self.product_price = max(0.1, self.np_random.normal(self.product_price_base,
self.price_std))
        self.demand = self._sample_demand()
        self.current_step = 0
        self.last_product_inventory_before_sale = 0
        self.last_sold_units = 0
        return self._get_obs(), {}

    def _get_obs(self):
        return np.array([self.raw_inventory, self.product_inventory,
                         self.raw_price, self.product_price, self.demand,
                         self.cash], dtype=np.float32)

    def step(self, action):
        if not self.action_space.contains(action):
            raise ValueError(f"Action must be 0, 1 or 2; got {action!r}")
        self.current_step += 1
        reward = 0.0
        invalid_action = False
        purchase_cost = 0.0
        served_demand = self.demand

        if action == 1:
            if self.cash >= self.raw_price:
                purchase_cost = float(self.raw_price)
                self.cash -= purchase_cost
                self.raw_inventory += 1
                reward -= purchase_cost
            else:
                reward -= self.invalid_action_penalty
                invalid_action = True
        elif action == 2:
            if self.raw_inventory >= 2:
```

```
                self.raw_inventory -= 2
                self.product_inventory += 1
                reward += self.conversion_bonus
            else:
                reward -= self.invalid_action_penalty
                invalid_action = True

        self.last_product_inventory_before_sale = self.product_inventory
        sold_units = min(self.product_inventory, served_demand)
        self.last_sold_units = sold_units
        revenue = float(sold_units * self.product_price)
        self.product_inventory -= sold_units
        self.cash += revenue
        reward += revenue

        unmet_demand = served_demand - sold_units
        reward -= self.stockout_penalty * unmet_demand
        holding_cost_total = self.holding_cost * (self.raw_inventory + self.product_inventory)
        reward -= holding_cost_total

        self.raw_price = max(0.1, self.raw_price + self.np_random.normal(0, self.price_std))
        self.product_price = max(0.1, self.product_price + self.np_random.normal(0, self.price_std))
        self.demand = self._sample_demand()  # next observation refers to NEXT period

        # Purchases require enough cash, so bankruptcy is not normally reachable.
        # Kept as a backward-compatible parameter, not an active reward penalty.
        terminated = self.cash <= 0
        truncated = self.current_step >= self.max_steps
        if terminated:
            reward -= self.bankruptcy_penalty

        info = {'sold_units': sold_units, 'unmet_demand': unmet_demand,
                'demand_served_period': served_demand, 'revenue': revenue,
                'purchase_cost': purchase_cost, 'holding_cost': holding_cost_total,
                'invalid_action': invalid_action,
                'product_inventory_before_sale': self.last_product_inventory_before_sale,
                'product_inventory_after_sale': self.product_inventory,
                'cash': self.cash}
        return self._get_obs(), float(reward), bool(terminated), bool(truncated), info

    def render(self):
        print(f"Step: {self.current_step}")
        print(f"Raw Inventory: {self.raw_inventory}")
        print(f"Product Inventory Before Sale: {self.last_product_inventory_before_sale}, After Sale: {self.product_inventory}")
        print(f"Raw Price: {self.raw_price:.2f}, Product Price: {self.product_price:.2f}")
        print(f"Next Period Demand: {self.demand}, Cash: {self.cash:.2f}")


```

```python
class NormalizeObservation(gym.ObservationWrapper):
    def __init__(self, env):
        super().__init__(env)
        self.scale_factors = np.array([10, 10, 10, 10, 1, 2000], dtype=np.float32)
        self.observation_space = spaces.Box(
            low=self.observation_space.low / self.scale_factors,
            high=self.observation_space.high / self.scale_factors,
            dtype=np.float32)

    def observation(self, obs):
        return (obs / self.scale_factors).astype(np.float32)


class ReplayBuffer:
    def __init__(self, capacity):
        self.buffer = deque(maxlen=capacity)

    def push(self, state, action, reward, next_state, terminated):
        self.buffer.append((np.array(state, copy=True), action, reward,
                            np.array(next_state, copy=True), terminated))

    def sample(self, batch_size):
        batch = random.sample(self.buffer, batch_size)
        states, actions, rewards, next_states, terminations = zip(*batch)
        return (np.asarray(states, dtype=np.float32), np.asarray(actions),
                np.asarray(rewards, dtype=np.float32),
                np.asarray(next_states, dtype=np.float32),
                np.asarray(terminations, dtype=np.float32))

    def __len__(self):
        return len(self.buffer)


class DQN(nn.Module):
    def __init__(self, input_dim, output_dim):
        super().__init__()
        self.network = nn.Sequential(nn.Linear(input_dim, 64), nn.ReLU(),
                                     nn.Linear(64, 64), nn.ReLU(),
                                     nn.Linear(64, 64), nn.ReLU(),
                                     nn.Linear(64, output_dim))

    def forward(self, x):
        return self.network(x)

```